UT2. MANEJO DE CONECTORES

ACTIVIDAD 7

Importación y consultas con procedimientos almacenados

Título: Importación de datos y consulta de precipitación media en MySQL con procedimiento almacenado.

Descripción:

En esta actividad, importarás una tabla de datos de precipitaciones en Algemesí desde un archivo CSV a MySQL, realizarás una consulta para obtener el valor medio de las precipitaciones durante el mes de octubre y ejecutarás un procedimiento almacenado para calcular la precipitación total. El objetivo es que practiques desde la importación de datos hasta la ejecución de procedimientos almacenados usando la interfaz CallableStatement en Java.

Objetivos:

- Importar datos desde un archivo CSV a una base de datos MySQL.
- Realizar consultas en MySQL para obtener datos agregados.
- Ejecutar un procedimiento almacenado desde una aplicación Java usando CallableStatement.
- Gestionar las excepciones SQL correctamente y mostrar los resultados en la consola.

Instrucciones:

1. Importación de datos desde CSV a base de datos MySQL:

- Crea una clase **CrearTablaClima** que cree en la base de datos **ut2** una tabla llamada **datos_climaticos**, como hiciste en la actividad 2. **[CREA LA TABLA DESDE EL PROGRAMA EN JAVA.]**
- Revisa los datos del archivo adjunto (**PrecipAlgemesiOctubre.csv**) y elige el tipo de cada columna. La tabla debe tener **exactamente las 6 columnas del archivo, en el mismo orden** (Provincia, Estacion, Fecha, Temperatura, Humedad, Precipitacion), sin columnas adicionales. La columna Fecha debe ser de tipo DATE.
- Ejecuta CrearTablaClima **una sola vez**.
- Importa el archivo CSV a la tabla **datos_climaticos** usando phpMyAdmin: selecciona la tabla → pestaña **Importar** → elige el archivo → formato **CSV**. En la opción de omitir líneas al principio del archivo, pon **1**, para no importar la fila de cabecera.
- Comprueba en phpMyAdmin que la tabla tiene **31 filas**.

2. **Consulta de la precipitación media mensual con Resultset**
   a. **Implementación de Consulta:**
      - Crea una segunda clase **ConsultaPrecipitaciones**. En ella, implementa un método que realice una consulta SQL para calcular la precipitación media (AVG) durante el mes de octubre (MONTH(Fecha) = 10) en la estación 'Algemesi'.
      - Usa el conector JDBC para conectarte a la base de datos y ejecutar la consulta SQL desde el programa (usando Statement).
   b. **Mostrar Resultados:**
      - Muestra en la consola el resultado con el mensaje: "*Precipitación media en octubre: [valor] mm*".

3. **Ejecución de un procedimiento almacenado:**

   a. Define el procedimiento almacenado
      - En MySQL crea un procedimiento almacenado llamado 'CalcularPrecipitacionTotal' que reciba como parámetro el nombre de la estación ('Algemesi') y devuelva la **precipitación total** para esa estación durante el mes de octubre.
      - Puedes definir el procedimiento almacenado con el siguiente esquema:

```
DELIMITER //

CREATE PROCEDURE CalcularPrecipitacionTotal(IN estacionNombre
VARCHAR(15), OUT totalPrecipitacion DECIMAL(10, 2))
BEGIN
    SELECT SUM(Precipitacion) INTO totalPrecipitacion
    FROM datos_climaticos
    WHERE Estacion = estacionNombre;
END //

DELIMITER ;
```

   b. Llamada al Procedimiento desde Java:
      - En tu aplicación Java, usa CallableStatement para llamar al procedimiento CalcularPrecipitacionTotal.
      - Define el parámetro de entrada estacionNombre y el parámetro de salida totalPrecipitacion.
      - Ejecuta el procedimiento y muestra el resultado de la precipitación total en la consola.

4. **Gestión de excepciones:**
   - Implementa un método muestraErrorSQL para mostrar detalles del error en caso de que ocurra una SQLException. Este método debe mostrar:

    - El mensaje de error (getMessage()).
    - El estado SQL (getSQLState()).
    - El código de error del proveedor (getErrorCode()).
- Captura cualquier otra excepción y muestra el mensaje correspondiente en la consola.

5. Prueba del programa:

- Ejecuta CrearTablaClima y comprueba en phpMyAdmin que se ha creado la tabla datos_climaticos vacía.
- Importa el CSV y comprueba que la tabla tiene 31 filas.
- Crea el procedimiento almacenado en phpMyAdmin.
- Ejecuta ConsultaPrecipitaciones y comprueba que se muestran la precipitación media y la precipitación total de octubre.
- Incluye una captura de cada paso.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Presenta la tarea, el código es correcto, pero no ha demostrado su funcionamiento mediante las capturas de pantalla
7	Presenta la tarea, el código es correcto, demuestra su funcionamiento mediante las capturas de pantalla, pero no describe los elementos principales.
9	Se completa la tarea correctamente en todos sus apartados
10	Se implementa una funcionalidad extra o un detalle innovador que mejora el programa.

Entrega:

1. **<u>Incluye aquí el código fuente</u>**

CLASE CrearTablaClima:

```
package org.example;
import java.sql.*;



public class CrearTablaClima {
    private static final String URL = "jdbc:mysql://localhost:3306/ut2";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        // Sentencia SQL para crear la tabla clientes (sin usar IF NOT EXISTS)
```

```
        String sql = "CREATE TABLE datos_climaticos ("
                + "Provincia VARCHAR(32) NOT NULL, "
                + "Estacion VARCHAR(32) NOT NULL, "
                + "Fecha DATE NOT NULL,"
                + "Temperatura DECIMAL(6.2) NOT NULL,"
                + "Humedad DECIMAL(6.2) NOT NULL,"
                + "Precipitacion DECIMAL(6.2) NULL"
                + ")";

        // try-with-resources: la conexión y el Statement se cierran solos al salir del bloque
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
             Statement statement = connection.createStatement()) {

            // Las sentencias DDL
            statement.execute(sql);
            System.out.println("¡Tabla 'datos_climaticos' creada correctamente en la base de datos!");

        } catch (SQLException e) {
            // Gestión de errores SQL utilizando el metodo requerido
            muestraErrorSQL(e);
        } catch (Exception e) {
            // Captura de cualquier otra excepción general
            System.err.println("Error inesperado: " + e.getMessage());
            e.printStackTrace(System.err);
        }
    }

    public static void muestraErrorSQL(SQLException e) {
        System.out.println("\n--- ERROR DE BASE DE DATOS (SQLException) ---");
        System.out.println("Mensaje de error: " + e.getMessage());
        System.out.println("Estado SQL (SQLState): " + e.getSQLState());
        System.out.println("Código de error del proveedor: " + e.getErrorCode());
        System.out.println("-----------------------------------------------\n");
    }
}
```

CLASE ConsultaPrecipitaciones:

```
package org.example;
import java.sql.*;

public class ConsultaPrecipitaciones {
    private static final String url = "jdbc:mysql://localhost:3306/ut2";
    private static final String user = "dam";
    private static final String password = "123456";

    public static void main(String[] args) {
        // 1. consulta de la precipitación media mensual con statement
        String consulta = "select avg(precipitacion) as media from datos_climaticos " +
                "where estacion = 'algemesi' and month(fecha) = 10;";

        try (Connection connection = DriverManager.getConnection(url, user, password);
```

```
            Statement statement = connection.createStatement();
            ResultSet resultset = statement.executeQuery(consulta)) {

            if (resultset.next()) {
                double media = resultset.getDouble("media");
                System.out.println("Precipitación media en octubre: " + media + " mm");
            }

        } catch (SQLException e) {
            muestraErrorSQL(e);
        }

        // 2. ejecución del procedimiento almacenado con callablestatement
        String procedimiento = "{call CalcularPrecipitacionTotal(?, ?)}";

        try (Connection connection = DriverManager.getConnection(url, user, password);
             CallableStatement CalcularPrecipitacionTotal = connection.prepareCall(procedimiento)) {

            // definir parámetro de entrada
            CalcularPrecipitacionTotal.setString(1, "algemesi");

            // registrar parámetro de salida
            CalcularPrecipitacionTotal.registerOutParameter(2, Types.DECIMAL);

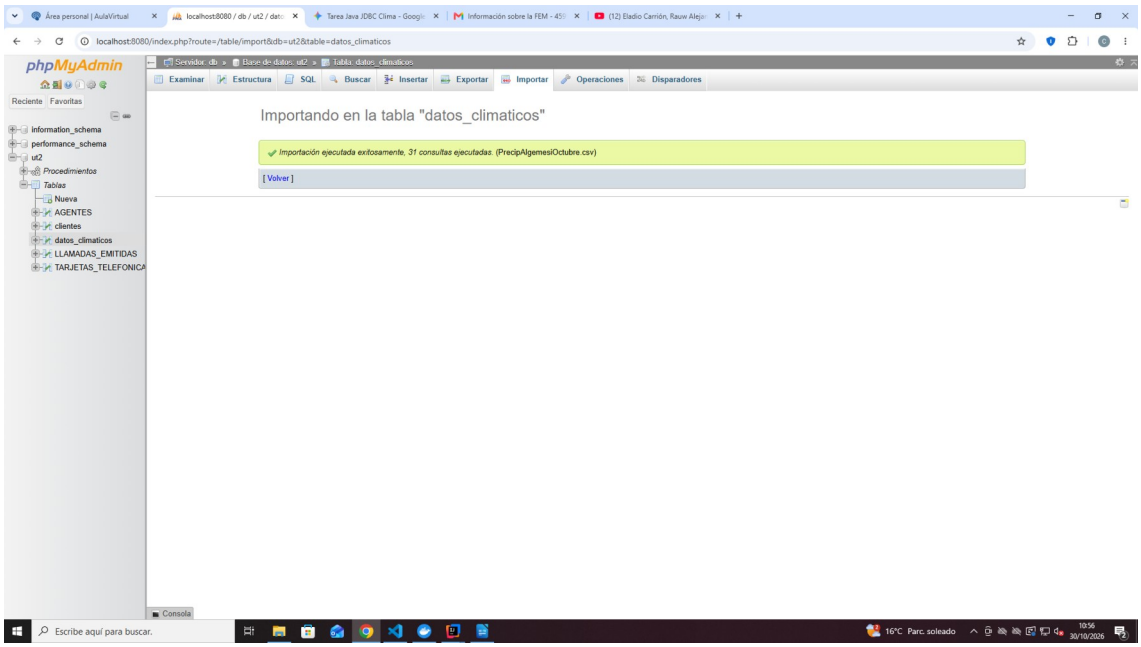
            // ejecutar el procedimiento
            CalcularPrecipitacionTotal.execute();

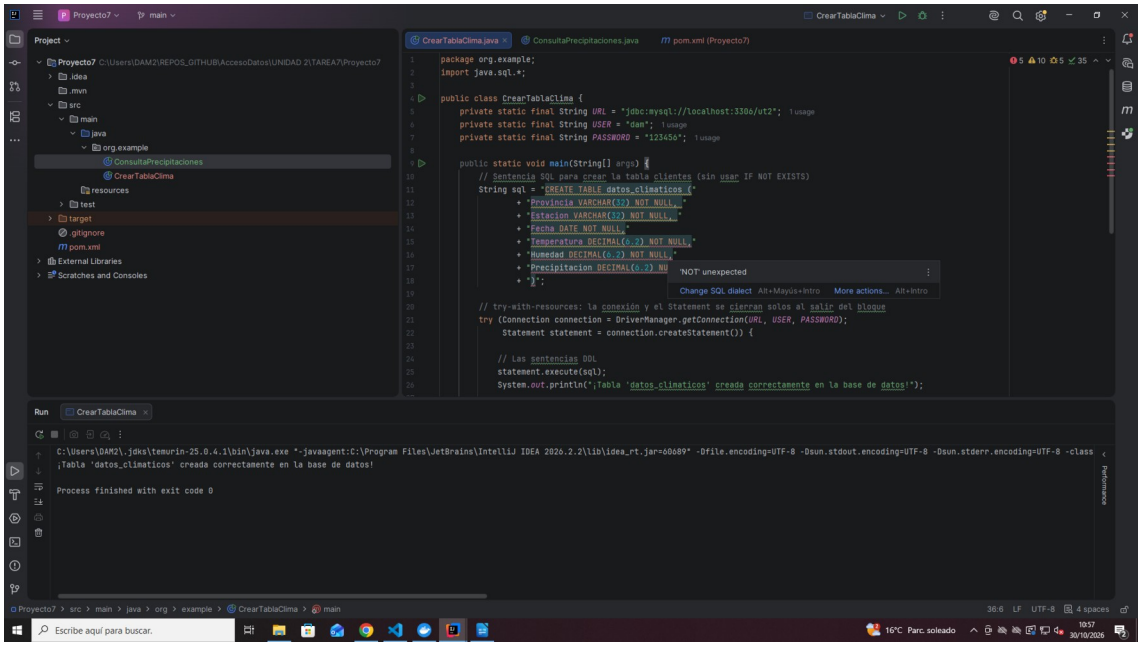
            // obtener el valor del parámetro de salida
            double totalPrecipitacion = CalcularPrecipitacionTotal.getDouble(2);
            System.out.println("precipitación total en octubre (procedimiento): " + totalPrecipitacion + "
mm");

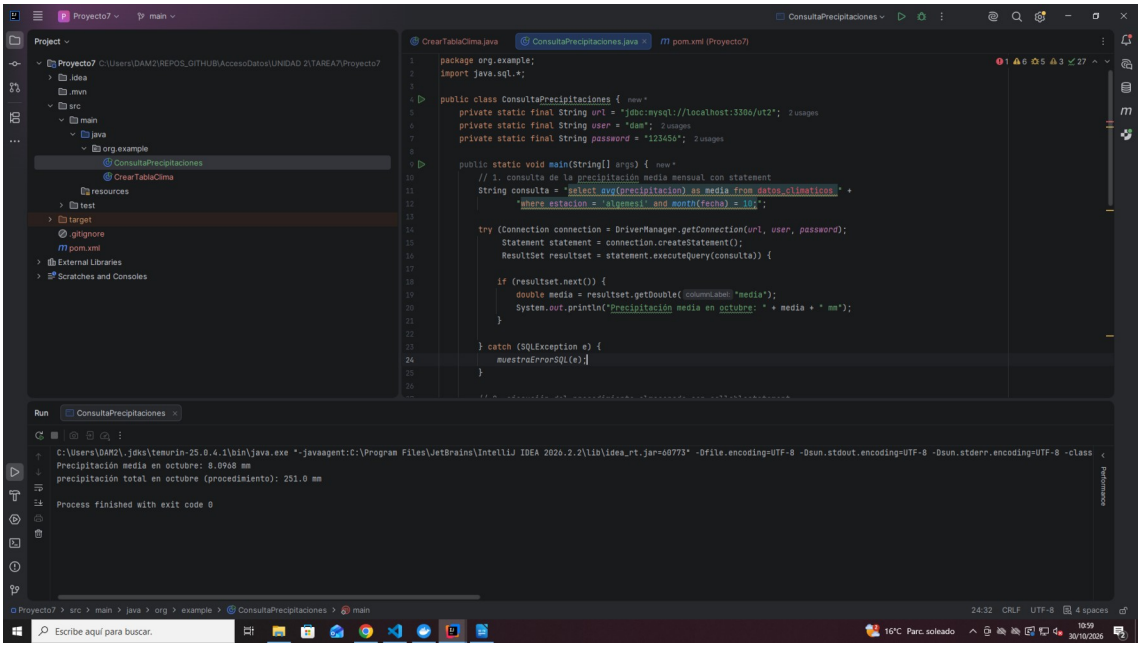
        } catch (SQLException e) {
            muestraErrorSQL(e);
        }
    }

    public static void muestraErrorSQL(SQLException e) {
        System.out.println("\n--- ERROR DE BASE DE DATOS (SQLException) ---");
        System.out.println("Mensaje de error: " + e.getMessage());
        System.out.println("Estado SQL (SQLState): " + e.getSQLState());
        System.out.println("Código de error del proveedor: " + e.getErrorCode());
        System.out.println("-----------------------------------------\n");
    }
}
```

2. **<u>Incluye aquí las capturas de pantalla que demuestren el correcto funcionamiento del programa</u>**:


phpMyAdmin
Importando en la tabla "datos_climaticos"
Importación ejecutada exitosamente, 31 consultas ejecutadas. (PrecipAlgemesiOctubre.csv)
[Volver]


CrearTablaClima.java
ConsultaPrecipitaciones.java
pom.xml (Proyecto7)
¡Tabla 'datos_climaticos' creada correctamente en la base de datos!
Process finished with exit code 0


ConsultaPrecipitaciones.java
Precipitación media en octubre: 8.0968 mm
precipitación total en octubre (procedimiento): 251.0 mm
Process finished with exit code 0

3. **Explica con tus palabras qué hace cada uno de estos elementos y por qué es necesario en el programa: los tipos de datos que has elegido para la tabla, AVG y MONTH en la consulta, CallableStatement, prepareCall, registerOutParameter y getDouble:**

Tipos de datos de la tabla:

- **VARCHAR**: Guarda texto usando solo el espacio que ocupa cada palabra.
- **DATE**: Guarda fechas reales para poder filtrar fácilmente por días, meses o años.
- **DECIMAL**: Guarda números con decimales exactos sin errores de redondeo .
- **AVG**: Calcula automáticamente la media de una columna de números en la base de datos.
- **MONTH**: Extrae el número del mes de una fecha para poder filtrar, por ejemplo, solo las de octubre.
- **CallableStatement:** Es la herramienta de Java diseñada específicamente para ejecutar procedimientos almacenados .
- **prepareCall:** Es la orden que le dice a la base de datos: "prepara la llamada a este procedimiento que voy a ejecutar".
- **registerOutParameter:** Sirve para avisarle a Java que el procedimiento nos va a devolver un dato de vuelta y de qué tipo será.
- **getDouble:** Es el comando para recoger ese dato de la base de datos y guardarlo en una variable numérica dentro de Java para poder usarlo o mostrarlo.